4.7 Programs and Processes – Role of Interrupts in Process State Transitions

Program vs Process

Program	Process
A passive entity – a set of instructions stored on disk (an executable file).	An active entity – a program in execution, along with its current activity (PC, registers, stack, allotted memory, resources).
Does not change state on its own.	Continuously moves between different states as it executes.

Process States

- **New:** process is being created.
- **Ready:** process has all resources except the CPU and is waiting to be scheduled.
- **Running:** process instructions are currently being executed by the CPU.
- **Waiting / Blocked:** process is waiting for some event (typically I/O completion) to occur.
- **Terminated:** process has finished execution.

Role of Interrupts in State Transitions

Interrupts are the fundamental mechanism that drives a process between states and enables the operating system to multiplex the CPU among several processes:

- **Running → Waiting:** When a running process executes an I/O request (e.g., a system call to read a file), it voluntarily gives up the CPU and enters the Waiting state until the I/O completes.
- **Waiting → Ready:** When the I/O device completes the operation, it raises a **hardware interrupt**. The interrupt handler (part of the OS) marks the corresponding process as Ready, making it eligible for scheduling again.
- **Running → Ready:** A **timer interrupt** (clock interrupt) at the end of a process's time slice forces the scheduler to preempt it, moving it from Running back to Ready so another process can run (time-sharing / round-robin scheduling).
- **Ready → Running:** The OS scheduler dispatches a ready process onto the CPU (this transition itself is usually initiated from within an interrupt/trap handler that decides to context-switch).
- **Any state → Terminated:** Completion of the program, or an exception (e.g., illegal memory access) forces early termination.

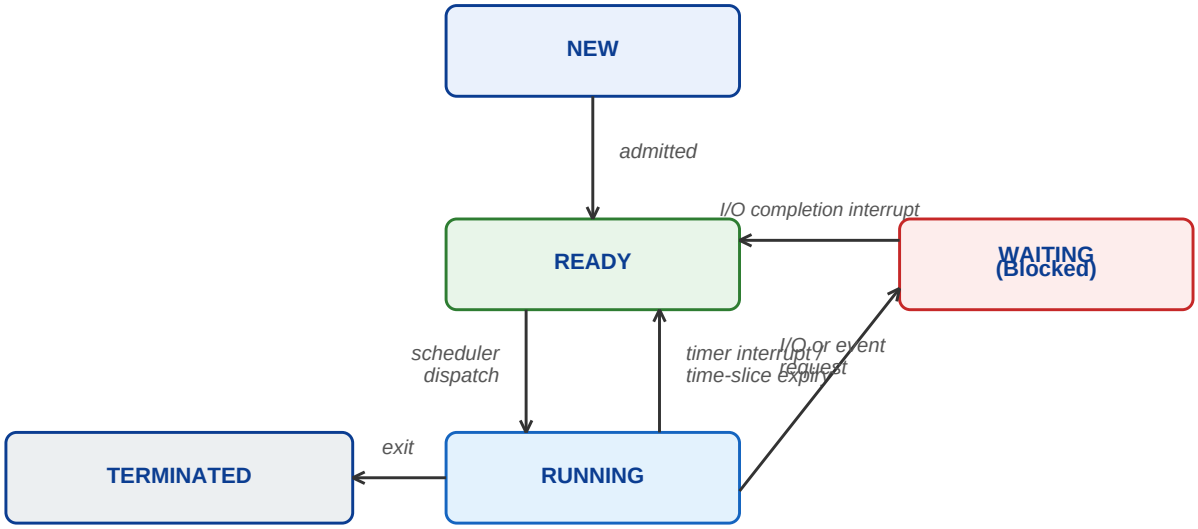

Fig: Process state transition diagram showing the role of interrupts

In short, without interrupts the CPU would have to continuously poll every device and process, wasting enormous processing time. Interrupts allow the CPU to remain productive and let the operating system react instantly to events, which is what makes multiprogramming / multitasking possible.